

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
IoT và ứng dụng

Đề tài : Hệ thống cảm biến đo
nhiệt độ, độ ẩm, ánh
sáng

Mục lục

I.	Mô tả hệ thống	3
1.	Mục đích.....	3
2.	Các thiết bị sử dụng trong hệ thống.....	3
3.	Các kiến thức cơ bản.....	4
3.1	ESP8266 NodeMCU 0.9	4
3.2	Giao thức MQTT.....	6
3.3	Websocket.....	8
II.	Thiết kế hệ thống.....	10
1.	Tổng quan hệ thống.....	Error! Bookmark not defined.
1.1	Sơ đồ hoạt động.....	Error! Bookmark not defined.
1.2	Mô tả sơ đồ.....	Error! Bookmark not defined.
2.	Database.....	16
2.1	Code tạo bảng.....	16
3.	Backend (Flask).....	18
3.1	Sơ đồ hoạt động của server.....	18
3.2	Mô tả hoạt động.....	18
3.3	Code chi tiết và giải thích.....	Error! Bookmark not defined.
4.	Client.....	Error! Bookmark not defined.
4.1	Kết nối với websocket.....	Error! Bookmark not defined.
4.2	Kết nối với database thông qua api.....	Error! Bookmark not defined.
5.	Arduino.....	25
5.1	Code chi tiết và giải thích.....	Error! Bookmark not defined.
5.2	Phần chưa làm được.....	Error! Bookmark not defined.
III.	Giao diện.....	Error! Bookmark not defined.
1.	Dashboard.....	Error! Bookmark not defined.
2.	Sensor Log.....	Error! Bookmark not defined.

I. Mô tả hệ thống

1. Mục đích.

Hệ thống sử dụng cảm biến để thông báo nhiệt độ và độ ẩm cho người dùng thông qua trang web có thể truy cập với mạng LAN. Trang web sẽ cung cấp nhiệt độ, độ ẩm và ánh sáng với dashboard hiển thị dễ nhìn, cho phép người dùng có thể xem thông tin nhiệt độ, độ ẩm và ánh sáng trực tiếp ở thời gian thực.

2. Các thiết bị sử dụng trong hệ thống.

Phần cứng

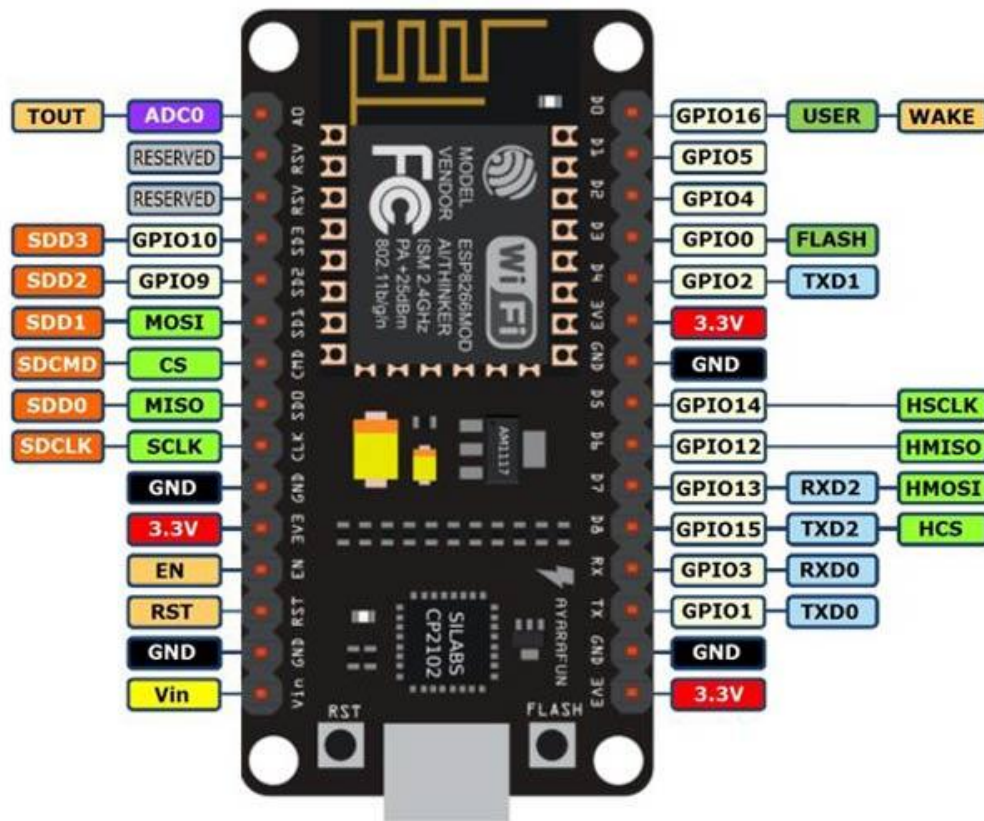
- 1 kit ESP8226 NodeMCU 0.9
- 1 cảm biến nhiệt độ và độ ẩm DHT22
- 1 cảm biến ánh sáng
- 3 đèn led
- 1 bảng mạch
- Và các loại dây dẫn đi kèm

Phần mềm:

- PlatformIO trên VSCode chạy trên nền Arduino
- Mosquitto server trên window
- Framework Flask + Flask-SocketIO (Backend)
- Framework ReactJS (Client)

3. Các kiến thức cơ bản.

3.1 ESP8266 NodeMCU 0.9



Giới thiệu chung.

ESP8266 là một vi mạch Wi-Fi giá rẻ với khả năng xử lý TCP/IP đầy đủ và khả năng vi điều khiển, được sản xuất bởi Espressif Systems. ESP8266 tích hợp phiên bản nâng cao của bộ xử lý 32-bit L106 Diamond của Tensilica và SRAM trên chip. Nó đã trở nên phổ biến nhờ vào giá thành hợp lý và tính dễ sử dụng.

Phần cứng.

Module Wi-Fi: Tích hợp Wi-Fi, hỗ trợ cả hai băng tần 2.4 GHz và 5 GHz.

Bộ xử lý: Bộ xử lý Tensilica L106 32-bit RISC chạy ở tốc độ 80 MHz.

Bộ nhớ:

- RAM: 80 KB
- Flash: 4 MB

Chân GPIO: 17 chân GPIO với 11 chân để giao tiếp với các thiết bị ngoại vi khác nhau.

ADC: Một chân ADC 10-bit cho đầu vào analog.

Nguồn điện: cổng micro-USB 3V

Các khả năng khác.

Framework hỗ trợ:

- Arduino
- ESP8266 Non-OS SDK
- ESP8266 RTOS SDK

Giao thức upload:

- Espota
- Esptool

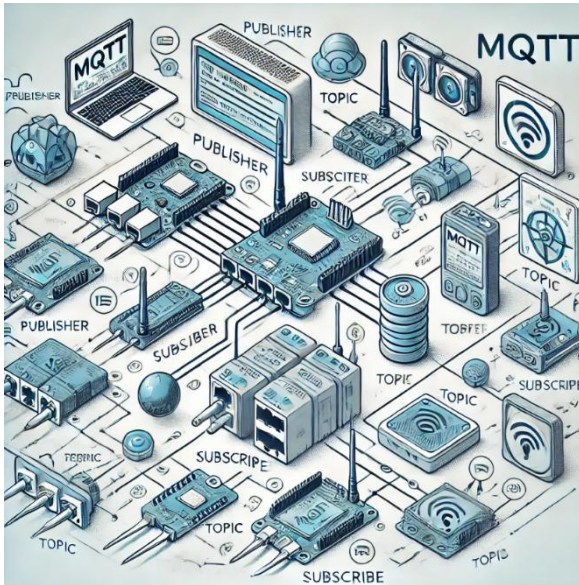
Tài liệu tham khảo.

- <https://docs.platformio.org/en/latest/boards/espressif8266/nodemcu.html>
- <https://nodemcu.readthedocs.io/en/latest/>
- <https://projecthub.arduino.cc/PatelDarshil/getting-started-with-nodemcu-esp8266-on-arduino-ide-b193c3>

3.2 Giao thức MQTT.

Giao thức MQTT (Message Queuing Telemetry Transport) là một giao thức nhắn tin nhẹ, phổ biến trong các ứng dụng IoT (Internet of Things) do tính đơn giản và hiệu quả của nó trong việc truyền dữ liệu giữa các thiết bị. Trong báo cáo này, chúng ta sẽ tìm hiểu cách sử dụng MQTT trong một hệ thống IoT dựa trên module ESP8266, một vi điều khiển tích hợp WiFi, thường được sử dụng trong các dự án IoT do tính năng kết nối mạng và chi phí thấp.

Tổng quan.



MQTT là giao thức nhắn tin dựa trên mô hình publish/subscribe, nơi các thiết bị phát dữ liệu (publishers) gửi thông điệp đến một hoặc nhiều chủ đề (topics), và các thiết bị nhận (subscribers) đăng ký các chủ đề này để nhận thông điệp. Giao thức này được thiết kế để hoạt động hiệu quả trong các môi trường có băng thông hạn chế hoặc kết nối không ổn định, với khả năng sử dụng rất ít tài nguyên hệ thống.

Các thành phần chính của MQTT.

- **Broker:** Là máy chủ trung gian quản lý các kết nối, truyền tải thông điệp giữa các publisher và subscriber. Broker phổ biến là Mosquitto.
- **Publisher:** Thiết bị hoặc ứng dụng gửi thông điệp lên broker.
- **Subscriber:** Thiết bị hoặc ứng dụng nhận thông điệp từ broker.

Chất lượng dịch vụ (QoS).

MQTT hỗ trợ ba mức độ QoS để đảm bảo thông điệp được truyền tải một cách chính xác:

- **QoS 0:** "At most once" - Thông điệp được gửi một lần mà không có đảm bảo việc nhận.
- **QoS 1:** "At least once" - Thông điệp được gửi ít nhất một lần, có thể bị trùng lặp.
- **QoS 2:** "Exactly once" - Thông điệp được gửi đúng một lần, không có khả năng bị trùng lặp.

ESP8266 và Vai trò trong Hệ thống IoT.

ESP8266 là một module WiFi phổ biến với khả năng kết nối mạng dễ dàng và chi phí thấp, lý tưởng cho các dự án IoT. Với vi xử lý 32-bit, nó cung cấp sức mạnh tính toán đủ để xử lý các nhiệm vụ IoT đơn giản và có thể lập trình bằng các ngôn ngữ như Arduino C++.

Triển khai MQTT với ESP8266.

Yêu cầu phần cứng:

- Module ESP8266 (ví dụ: NodeMCU)
- Cảm biến (ví dụ: cảm biến nhiệt độ DHT22)
- Nguồn cấp điện (USB hoặc pin)

Yêu cầu phần mềm:

- Arduino
- Thư viện PubSubClient để hỗ trợ MQTT
- Thư viện ESP8266WiFi để kết nối mạng

Các bước triển khai:

Bước 1: Cài đặt Extension PlatformIO trên VSCode và thêm ESP8266 vào Board Manager

- Mở PlatformIO, vào mục **PIO Home** -> **Open** -> **New Project**
- Đặt tên project tại **Project Name**
- Click vào **Board** -> Tìm kiếm **ESP8266**, sau đó cài đặt.

Bước 2: Cài đặt các thư viện cần thiết

Cấu hình file **platformio.ini**

```
[env:nodemcu]
platform = espressif8266
board = nodemcu
framework = arduino
monitor_port = COM3
monitor_speed = 115200
lib_deps =
    knolleary/PubSubClient@2.8
    adafruit/DHT sensor library@1.4.3
    adafruit/Adafruit Unified Sensor@1.1.9
```

- Cài đặt thư viện PubSubClient
- Cài đặt thư viện DHT để nhận đọc dữ liệu cảm biến
- Cài đặt thư viện ESP8266WiFi để kết nối WiFi.

Bước 3: Viết mã kết nối và gửi dữ liệu qua MQTT

Các đoạn mã kết nối sẽ được áp dụng trực tiếp trong dự án

Bước 4: Biên dịch, nạp mã và giám sát kết quả

- Kết nối ESP8266 với máy tính, chọn đúng board và port trong platformIO , sau đó upload mã.
- Mở Serial Monitor để giám sát kết nối và dữ liệu được gửi đi.

Kết luận.

Giao thức MQTT cùng với module ESP8266 cung cấp một giải pháp mạnh mẽ và linh hoạt cho các ứng dụng IoT, cho phép truyền dữ liệu giữa các thiết bị một cách hiệu quả và tin cậy. Với sự hỗ trợ của cộng đồng và các thư viện mã nguồn mở, việc triển khai hệ thống IoT với ESP8266 và MQTT trở nên dễ dàng và tiết kiệm chi phí.

Tài liệu tham khảo.

<https://www.emqx.com/en/mqtt-guide>

<https://www.hivemq.com/mqtt/>

<https://www.comparitech.com/net-admin/what-is-mqtt/>

3.3 Websocket.

Tổng quan.

WebSocket là một giao thức truyền tải dữ liệu hai chiều (full-duplex) trên web, cho phép truyền tải dữ liệu trong thời gian thực giữa trình duyệt web và máy chủ. Điều này có nghĩa là một kết nối duy trì giữa máy khách và máy chủ, cho phép thông tin được gửi đi và nhận lại một cách hiệu quả và nhanh chóng mà không cần phải thiết lập kết nối mới mỗi khi truyền tải thông tin.

Các sự kiện chính trong WebSocket:

1. **Mở kết nối (Open):** Khi một kết nối WebSocket được thiết lập giữa trình duyệt và máy chủ.
2. **Nhận dữ liệu (Message):** Khi trình duyệt hoặc máy chủ nhận được dữ liệu mới.
3. **Gửi dữ liệu (Send):** Khi trình duyệt hoặc máy chủ gửi dữ liệu.
4. **Đóng kết nối (Close):** Khi kết nối WebSocket bị đóng.
5. **Lỗi (Error):** Nếu có lỗi trong quá trình thiết lập hoặc sử dụng kết nối WebSocket.

Công dụng của WebSocket:

- **Ứng dụng web thời gian thực:** WebSocket thường được sử dụng để phát triển các ứng dụng web thời gian thực như trò chơi trực tuyến, ứng dụng trò chuyện và các ứng dụng cập nhật dữ liệu trực tiếp.
- **Hiệu suất cao:** WebSocket có hiệu suất cao hơn và tiết kiệm năng lượng hơn so với các giải pháp truyền tải dữ liệu trực tiếp khác như Ajax hoặc Comet.

Triển khai với Flask

Bước 1: Cài đặt thư viện

Tạo file **requirements.txt**

Sau đó thêm các thư viện sau vào file

```
eventlet
Flask
flask-mqtt
flask-socketio
flask-cors
flasgger
SQLAlchemy
```

Chạy lệnh **pip install -r requirements.txt**

Bước 2: Tạo máy chủ flask và thiết lập Websocket

Thiết lập có trong phần [Code chi tiết và giải thích backend](#)

Bước 3: Chạy máy chủ

Bước 4: Kết nối tới máy chủ websocket từ Client

Thiết lập có trong phần [Client](#)

Tài liệu tham khảo.

[Websocket là gì? Ưu và nhược điểm của websocket bạn cần biết \(stringee.com\)](#)

[WebSocket là gì? Lý do sử dụng WebSocket \(200lab.io\)](#)

[Socket là gì? WebSocket là gì? Hiểu hơn về Websocket | TopDev](#)

[ws - npm \(npmjs.com\)](#) (Triển khai)

II. Tính năng

1. Kết nối và quản lý cơ sở dữ liệu SQLite

- Thiết lập kết nối tới cơ sở dữ liệu SQLite.
- Chèn, cập nhật, xóa và truy vấn dữ liệu từ các bảng trong cơ sở dữ liệu.

2. Xử lý và quản lý dữ liệu cảm biến và hành động

- Nhận dữ liệu từ các cảm biến thông qua MQTT và lưu vào cơ sở dữ liệu.
- Lấy dữ liệu cảm biến từ cơ sở dữ liệu theo nhiều tiêu chí khác nhau (tất cả, theo trang, theo thời gian, theo tìm kiếm).
- Lấy dữ liệu hành động từ cơ sở dữ liệu theo nhiều tiêu chí khác nhau (tất cả, theo trang, theo thời gian, theo tìm kiếm).

3. Cung cấp API cho giao diện người dùng

- Cung cấp API RESTful để lấy thông tin tổng quan về dữ liệu cảm biến và hành động.
- Cung cấp API để lấy dữ liệu cảm biến và hành động theo trang, theo khoảng thời gian, theo tiêu chí tìm kiếm, và theo sắp xếp.

4. Giao tiếp thời gian thực qua WebSocket

- Thiết lập WebSocket để truyền dữ liệu thời gian thực từ máy chủ tới client.
- Gửi và nhận dữ liệu cảm biến và hành động qua WebSocket.

5. Giao tiếp với MQTT Broker

- Kết nối tới MQTT Broker để nhận và gửi thông báo MQTT.
- Nhận dữ liệu cảm biến từ MQTT và phát thông báo tới tất cả các client kết nối qua WebSocket.
- Nhận và xử lý thông báo hành động từ MQTT và phát tới tất cả các client kết nối qua WebSocket.

6. Xử lý dữ liệu đầu vào từ client

- Xử lý yêu cầu từ client để lấy dữ liệu cảm biến và hành động.
- Phát thông báo tới tất cả các client kết nối khi có dữ liệu mới từ MQTT.

7. Quản lý kết nối client

- Quản lý kết nối và ngắt kết nối của các client.
- Phát thông báo tới tất cả các client khi có sự thay đổi trong dữ liệu cảm biến hoặc hành động.

Home Page

Dashboard

Action History

Data Sensor

HELLO NGUYỄN GIANG NAM

Action History

Row: 50

Is Latest: True

Từ Ngày 10/09/2024

Đến Ngày 10/09/2024

Profile	#	Device 	Topic	Command	Status	Timestamp  Filter Timestamp
	1	ALLDEVICE	allDevice/sub	ON	TurnOn Successfully	2024-10-09 12:44:47
	2	ALLDEVICE	allDevice/sub	ON	TurnOn Successfully	2024-10-09 12:44:47
	3	AIRCONDITIONER	airConditioner/sub	ON	TurnOn Successfully	2024-10-09 12:44:46
	4	AIRCONDITIONER	airConditioner/sub	ON	TurnOn Successfully	2024-10-09 12:44:46
	5	ALLDEVICE	allDevice/sub	OFF	TurnOff Successfully	2024-10-09 12:44:45
	6	ALLDEVICE	allDevice/sub	OFF	TurnOff Successfully	2024-10-09 12:44:45
	7	LIGHTBULB	lightBulb/sub	OFF	TurnOff Successfully	2024-10-09 12:44:44
	8	LIGHTBULB	lightBulb/sub	OFF	TurnOff Successfully	2024-10-09 12:44:44
	9	ALLDEVICE	allDevice/sub	ON	TurnOn Successfully	2024-10-09 12:44:44
	10	ALLDEVICE	allDevice/sub	ON	TurnOn Successfully	2024-10-09 12:44:44
	11	LIGHTBULB	lightBulb/sub	ON	TurnOn Successfully	2024-10-09 12:44:43
	12	LIGHTBULB	lightBulb/sub	ON	TurnOn Successfully	2024-10-09 12:44:43
	13	AIRCONDITIONER	airConditioner/sub	ON	TurnOn Successfully	2024-10-09 12:44:42
	14	AIRCONDITIONER	airConditioner/sub	ON	TurnOn Successfully	2024-10-09 12:44:42
	15	FAN	fan/sub	ON	TurnOn Successfully	2024-10-09 12:44:41

<

1

2

3

>

3. Giao diện ghi lại dữ liệu Sensor theo thời gian thực - Data Sensor

Home Page

Dashboard

Action History

Data Sensor

Profile

HELLO NGUYỄN GIANG NAM

Data Sensor

Row: 50

Is Latest: True

12:48:37

Từ Ngày 10/09/2024

Đến Ngày 10/09/2024

Mode Real Time Is On

#	Nhiệt Độ Filter Temperature	Độ Ẩm Filter Humidity	Ánh Sáng Filter Light	Thời Gian Filter Timestamp
1	30.5	60.5	105	2024-10-09 12:48:35
2	30.5	60.5	105	2024-10-09 12:48:35
3	30.5	60.4	106	2024-10-09 12:48:33
4	30.5	60.4	106	2024-10-09 12:48:33
5	30.5	60.5	106	2024-10-09 12:48:31
6	30.5	60.5	106	2024-10-09 12:48:31
7	30.5	60.5	106	2024-10-09 12:48:29
8	30.5	60.5	106	2024-10-09 12:48:29
9	30.5	60.5	105	2024-10-09 12:48:27
10	30.5	60.5	105	2024-10-09 12:48:27
11	30.5	60.5	106	2024-10-09 12:48:25
12	30.5	60.5	106	2024-10-09 12:48:25
13	30.5	60.6	106	2024-10-09 12:48:23
14	30.5	60.6	106	2024-10-09 12:48:23

1

2

3

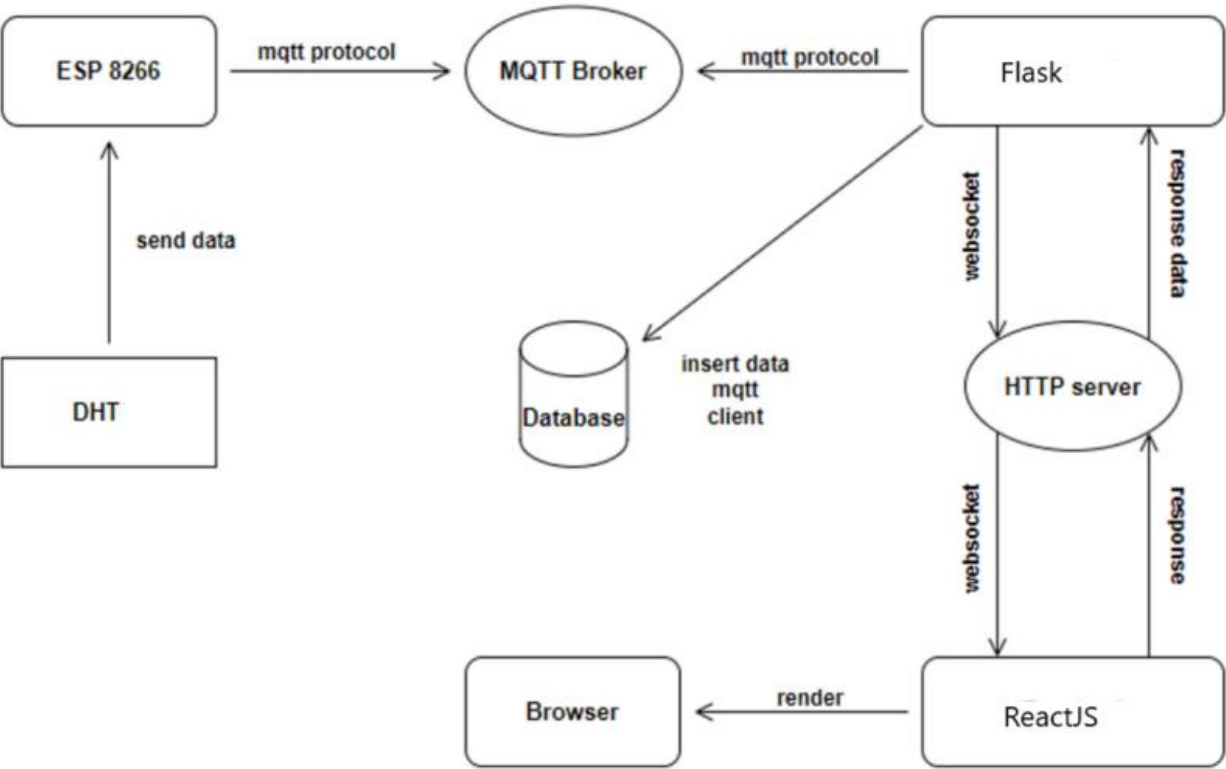
...

171

IV. Thiết kế hệ thống.

1. Tổng quan hệ thống

Tổng quan sơ đồ hoạt động



MQTT Broker:

- Kết nối với mqtt://localhost:1883.
- Cần thông tin đăng nhập với username: **ngn**, password: **ngn**

Flask Server (HTTP server):

- Lắng nghe trên cổng 3000.
- Sử dụng các route để xử lý yêu cầu từ client (/api/v1/sensor/* và /api/v1/device/.*).

WebSocket:

- Được tạo trên cùng một máy chủ Flask, sử dụng **eventlet** để quản lý luồng.
- Lắng nghe kết nối WebSocket từ client.
- Cho phép hiển thị thông tin cảm biến realtime.

SQLite Database:

- CSDL là 1 file nằm trên cùng hệ thống thư mục gốc của backend server
- Các bảng dữ liệu bao gồm **DataRealTime** và **DeviceHistory**.

Web Client:

- Hiển thị realtime đo lường của cảm biến DHT22
- Hiển thị thông tin lịch sử đo cảm biến
- Hiển thị thông tin lịch sử điều khiển thiết bị.
- Lọc/Tìm kiếm dữ liệu linh hoạt

2. Thiết Kế Database

Hệ thống gồm 2 bảng được sử dụng để lưu trữ dữ liệu của Sensor và lịch sử điều khiển device lần lượt là:

Table for DataRealTime

```
CREATE TABLE `DataRealTime` (  
    `id` INT NOT NULL AUTO_INCREMENT,  
    `temp` VARCHAR(255) NOT NULL,  
    `humidity` INT NOT NULL,  
    `light` INT NOT NULL,  
    `timestamp` DATETIME NULL DEFAULT NULL,  
    PRIMARY KEY (`id`))
```

Table for DeviceHistory

```
CREATE TABLE `DeviceHistory` (  
    `id` INT NOT NULL AUTO_INCREMENT,  
    `device_name` VARCHAR(255) NOT NULL,  
    `command` VARCHAR(255) NOT NULL,  
    `status` VARCHAR(255) NOT NULL,  
    `timestamp` DATETIME NULL DEFAULT NULL,  
    PRIMARY KEY (`id`))
```

DataRealTime	
id 	integer
temp	float
humidity	float
DeviceHistory	
id 	integer
device_name	varchar
command	varchar
status	varchar
timestamp	datetime

```

'id' INT NOT NULL AUTO_INCREMENT,

'device_name' VARCHAR(50) NOT NULL,

'command' VARCHAR(50) NOT NULL,

'status' VARCHAR(50) NOT NULL,

'timestamp' DATETIME NULL DEFAULT NULL,

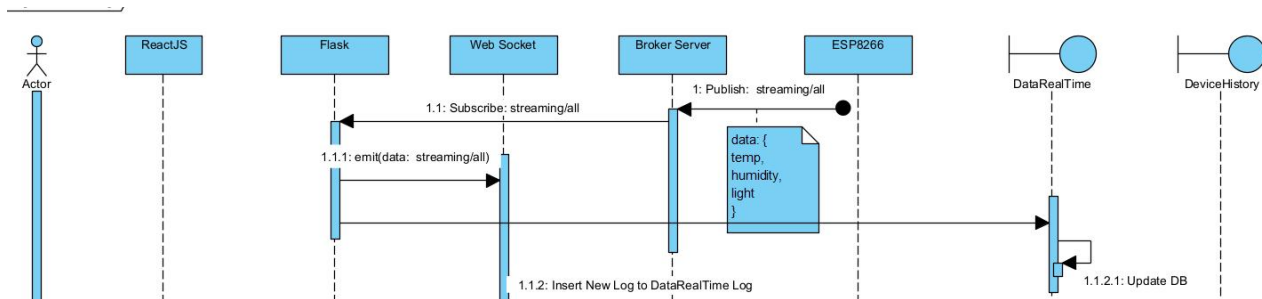
PRIMARY KEY ('id'))

```

3. Mô tả sơ đồ luồng hoạt động

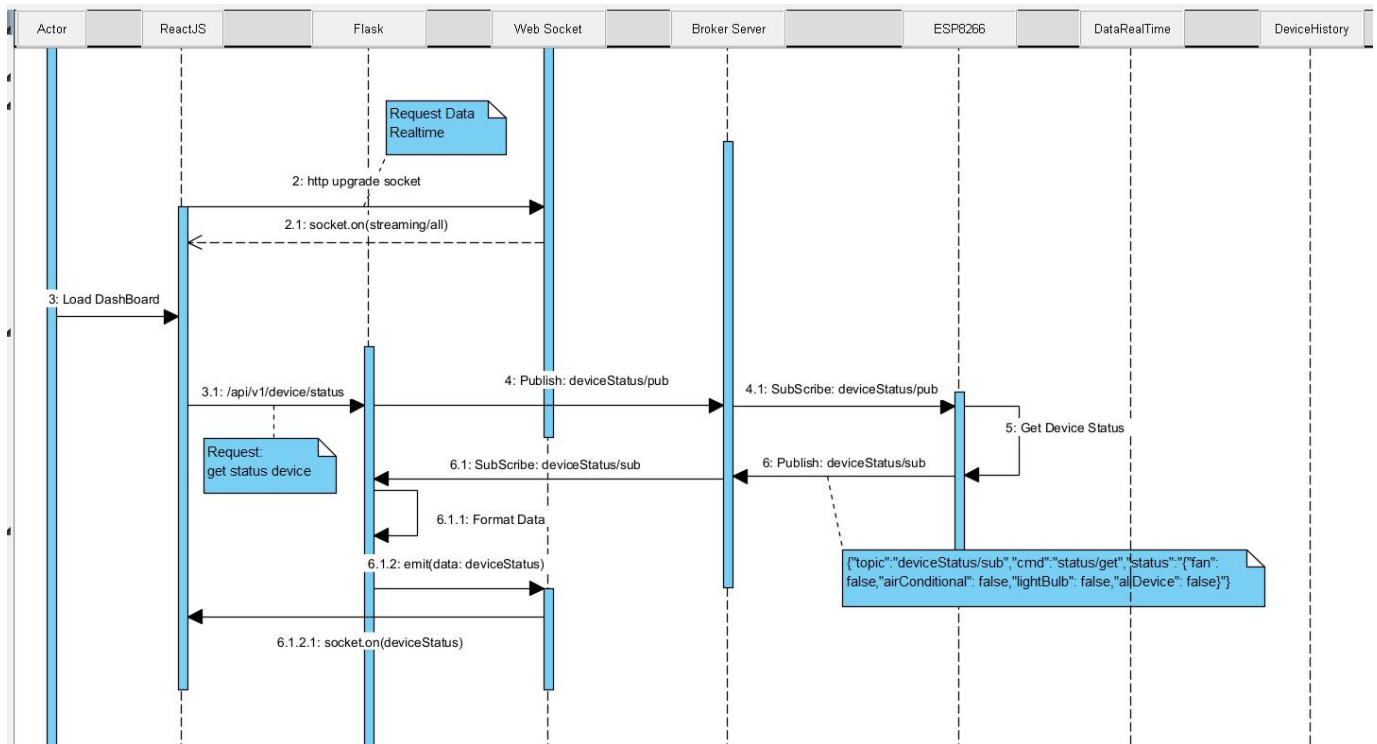
3.1 Luồng 1: Khởi tạo luồng Sensor

Dữ liệu Sensor từ ESP về Flask, dữ liệu đến Flask được lưu vào DB với tên bảng là DATAREALTIME và đổ về Web Socket

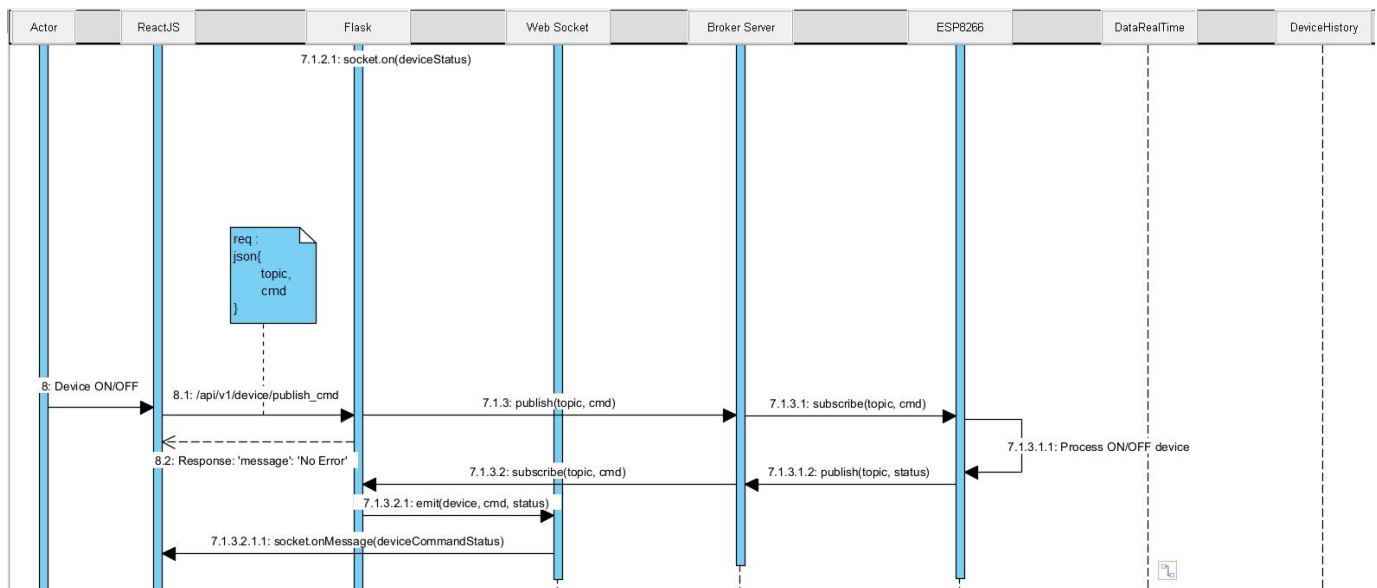


3.2 Luồng 2: Truy cập vào trang DashBoard

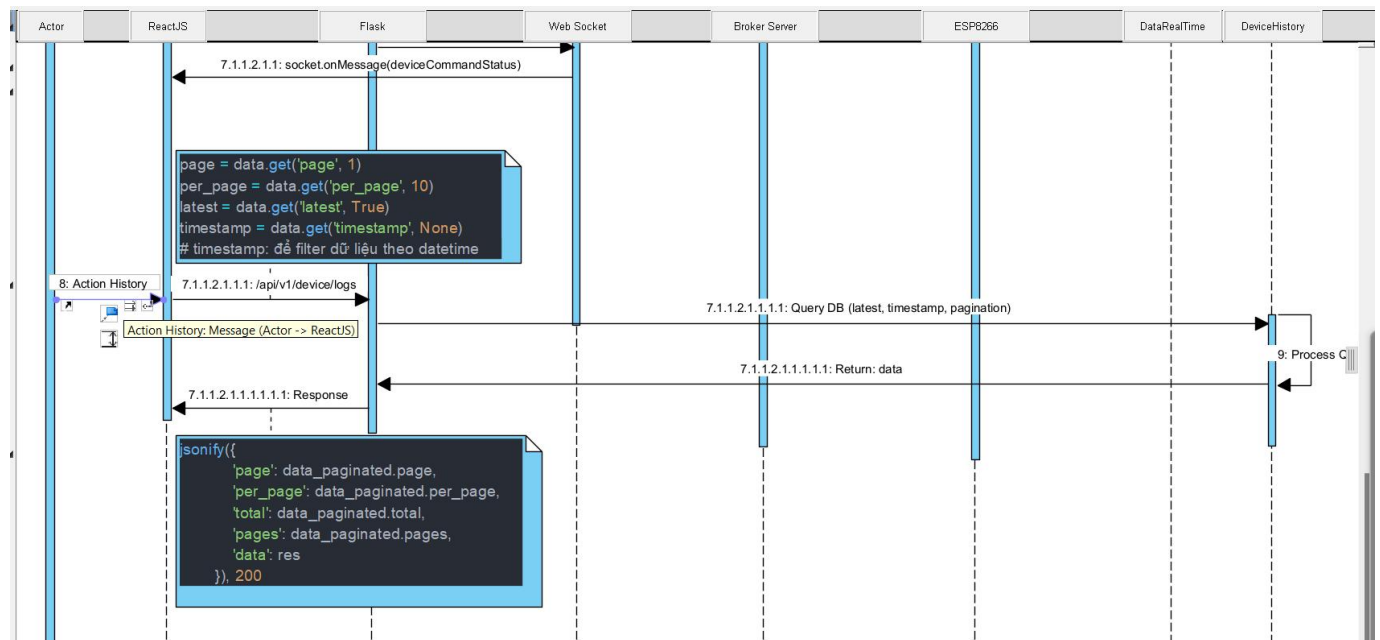
- Nhận dữ liệu Sensor qua WebSocket
- Nhận trạng thái ban đầu của device qua web socket



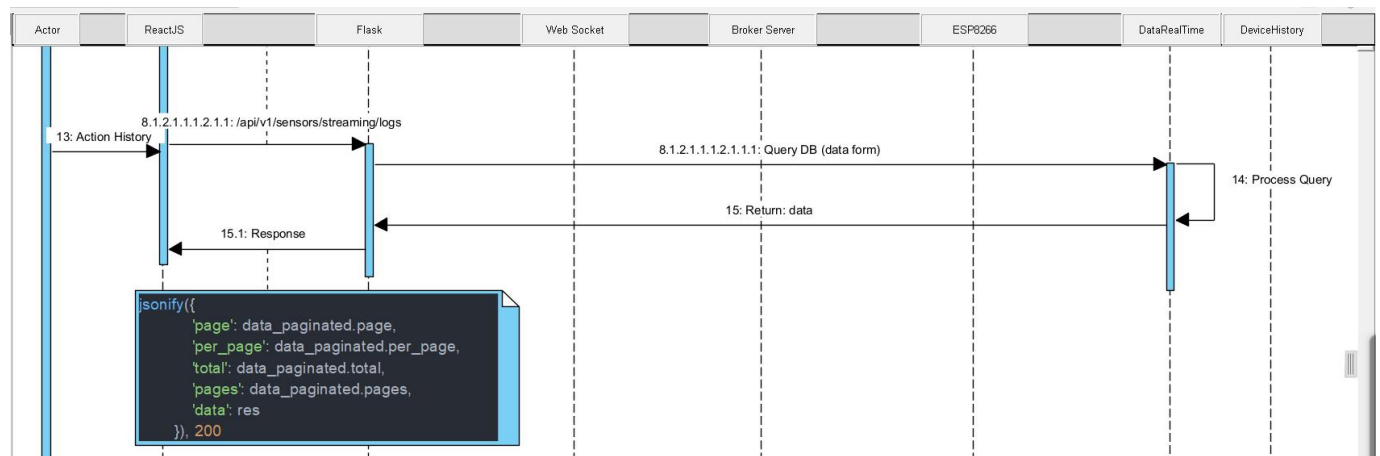
3.3 Luồng 3:Điều khiển Device



3.4 Luồng 4: Action History



3.5 Luồng 5: Data Sensor



4. API Document

Chi tiết tại: [API Documentation \(getpostman.com\)](https://getpostman.com)

V. Source Code.

1. Database.

Database được sử dụng là SQLite gồm 2 bảng:

DataRealTime : Dùng để ghi lại các bản ghi (logs) nhận được từ cảm biến DHT22

DeviceHistory: Dùng để ghi lại các bản ghi bật tắt thiết bị từ phía client

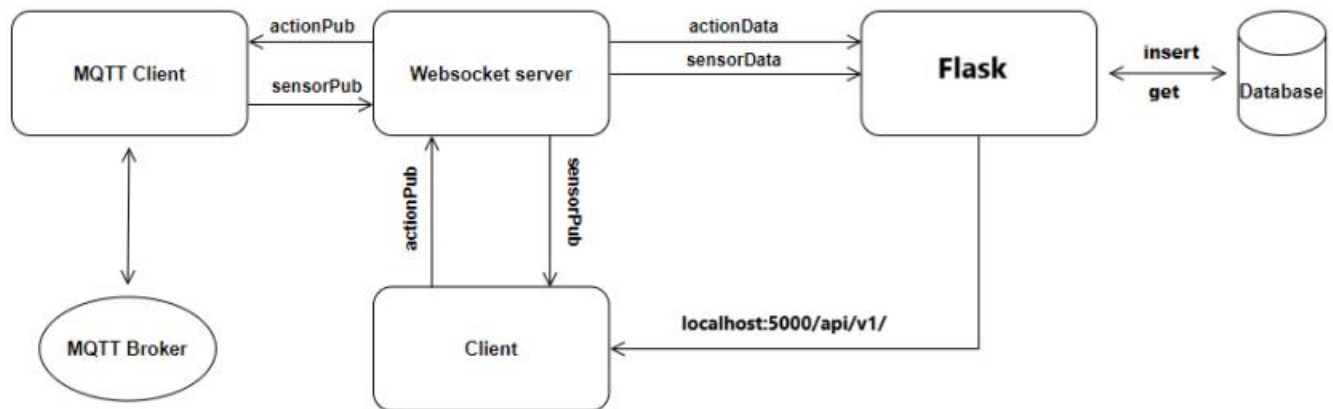
Code tạo bảng.

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime
```

```
db = SQLAlchemy()
class DataRealTime(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    temp = db.Column(db.Integer, nullable=False)
    humidity = db.Column(db.Integer, nullable=False)
    light = db.Column(db.Integer, nullable=False)
    timestamp = db.Column(db.DateTime(50), nullable=False)
    def to_dict(self):
        return {
            'id': self.id,
            'temp': self.temp,
            'humidity': self.humidity,
            'light': self.light,
            'timestamp': self.timestamp
        }
class DeviceHistory(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    device_name = db.Column(db.String(50), nullable=False)
    command = db.Column(db.String(50), nullable=False)
    status = db.Column(db.String(50), nullable=False)
    timestamp = db.Column(db.DateTime(50), nullable=False)
    def to_dict(self):
        return {
            'id': self.id,
            'device_name': self.device_name,
            'command': self.command,
            'status': self.status,
            'timestamp': self.timestamp.strftime("%Y-%m-%d %H:%M:%S")
        }
```

2. Backend (Flask).

2.1 Sơ đồ hoạt động của server.



2.2 Mô tả hoạt động.

Nhận dữ liệu từ MQTT:

- MQTT Client kết nối đến MQTT broker và đăng ký chủ đề `sensorPub` và `actionPub`.
- Khi nhận được tin nhắn từ `sensorPub`, MQTT Client chèn dữ liệu vào bảng `sensor`.
- Khi nhận được tin nhắn từ bất kỳ chủ đề nào, MQTT Client phát lại tin nhắn đến tất cả các client WebSocket.

Gửi dữ liệu đến WebSocket:

- Client kết nối đến WebSocket server.
- Khi client gửi tin nhắn, WebSocket server chèn dữ liệu vào bảng `action` và phát lại tin nhắn đến MQTT broker.

Xử lý yêu cầu HTTP:

- Flask server lắng nghe các yêu cầu HTTP từ client.
- Dựa vào route, các controller sẽ gọi các hàm từ model để thao tác dữ liệu.
- Kết quả được trả về cho client thông qua HTTP response.

2.3 Khai báo các Topic có trên hệ thống

```
topics_publish = {
  'fan': 'fan/pub',
  'airConditioner': 'airConditioner/pub',
  'lightBulb': 'lightBulb/pub',
  'allDevice': 'allDevice/pub',
  'deviceStatus': 'deviceStatus/pub',
}
topics_subscribe = {
  'streaming/all': 'streaming/all',

  'temperature': 'temperature',
  'humidity': 'humidity',
  'lights': 'lights',
  'allDevice': 'allDevice/sub',
  'deviceStatus': 'deviceStatus/sub',

  'fan': 'fan/sub',
  'airConditioner': 'airConditioner/sub',
  'lightBulb': 'lightBulb/sub'
}
commands = {
  'turnOn': 'ON',
  'turnOff': 'OFF',
}

sensors = {
  'temp' : {
    'label': 'Nhiệt Độ',
    'threshold': 50
  },
  'humidity' : {
    'label': 'Độ Ẩm',
    'threshold': 80
  },
  'light': {
    'label': 'Ánh Sáng',
    'threshold': 500
  },
  'dust' : {
    'label': 'Độ Bụi',
    'threshold': 50
  },
  'windSpeed' : {
    'label': 'Tốc Độ Gió',
    'threshold': 10
  },
  'rain' : {
    'label': 'Mức Mưa',
    'threshold': 50
  }
}
```

1. topics_publish: định nghĩa cho các topic Publish đến Broker Server
2. Topics_subscribe: định nghĩa cho các topic subscribe từ Broker Server
3. Commands: định nghĩa ra các lệnh để điều khiển device
4. Sensors: định nghĩa ra 1 tập hợp các sensor gồm nhãn và ngưỡng

2.4 Khởi tạo server

```
''' -----Load Variable----- '''

# Global variable to keep track of the publish status
publish_status = {'status': None, 'message': None}
front_end = os.getenv('FRONTEND_URL')
''' -----Server Config----- '''
app = Flask(__name__)
CORS(app, resources={r"/*": {"origins": "*"}}, supports_credentials=True)
app.config.from_object(ApplicationConfig)
db.init_app(app)
SOCKET_URL = os.getenv('SOCKET_URL')
PORT = os.getenv('PORT')
mqtt = Mqtt(app)
socketio = SocketIO(app, cors_allowed_origins=front_end)
```

2.5 Xử lý Subscribe

```
@mqtt.on_message()
def handle_mqtt_message(client, userdata, msg):
    t = time.localtime()
    current_time = time.strftime("%H:%M:%S %Y-%m-%d", t) # Chuỗi thời gian
    try:
        message = json.loads(msg.payload.decode())
        print("mess_x: %s" % message)

        custom_data = None
        topic = msg.topic
        if topic in ['streaming/all', 'temperature', 'humidity', 'lights']:
            print("Emit real time data: ", topic)
            custom_data = {
                'topic': msg.topic,
                'message': {
                    'time': current_time,
                    'temp': message['temp'],
                    'humidity': message['humidity'],
                    'light': message['light'],
                    'dust': message['dust'],
                    'windSpeed': message['windSpeed'],
                    'rain': message['rain']
                }
            }
            ''' emit real time data '''
            socketio.emit(topic, custom_data)

            with app.app_context():
                timestamp = datetime.strptime(custom_data['message']['time'],
"%H:%M:%S %Y-%m-%d")
                new_data = DataRealTime(temp=custom_data['message']['temp'],
                                         humidity=custom_data['message']['humidit
y'],
                                         light=custom_data['message']['light'],
                                         timestamp=timestamp)
                db.session.add(new_data)
                db.session.commit()
        else:
            device_name = message['topic'].split('/')[0]
            cmd = message['cmd']
```

- Kiểm tra nếu topic subscribe về là topic được định nghĩa trong mảng thì dữ liệu đồ về cho WebSocket

- Sau đó lưu lại dữ liệu sensor vào database

- Nếu topic không nằm trong mảng trên, nghĩa

```

status = message['status']
note = None
if 'Successfully' in status:
    note = status
elif 'already off!' in status:
    note = device_name + ' is already off!'
elif 'already on!' in status:
    note = device_name + ' is already on!'
else:
    note = 'Some things went wrong'
print(note)
socketio.emit('device', {
    'device_name': device_name,
    'cmd': cmd,
    'status': note,
    'timestamp': current_time # timestamp for real time data
})

# Cập nhật Log Action
with app.app_context():
    timestamp = datetime.strptime(current_time, "%H:%M:%S %Y-%m-%d")
    new_log = DeviceHistory(device_name=topic, command=cmd,
status=status, timestamp=timestamp)
    db.session.add(new_log)
    db.session.commit()

except Exception as e:
    try:
        ''' Nhận trạng thái của device '''
        message = msg.payload.decode().replace('"status":{"', '"status:{"')
        message = message.replace('}"}', '}}}')
        message = json.loads(message)
        print(message)
        device_name = message['topic'].split('/')[0]
        cmd = message['cmd']
        socketio.emit('deviceStatus', {
            'device_name': device_name,
            'cmd': cmd,
            'status': message['status'],
            'timestamp': current_time # timestamp for real time data
        })
    except:
        print("Error processing message: ", e)

        print(f"Invalid topic: {msg.topic}")

```

là các topic này thuộc về các lệnh điều khiển device được publish khi client gọi qua API `/api/v1/device/publish_cmd`

2.6 Lấy trạng thái ban đầu của device

```

@app.route('/api/v1/device/status', methods=['GET'])
def device_status():
    try:
        topic = topics_publish['deviceStatus']
        cmd = 'status/get'
        mqtt.publish(topic, cmd)
        time.sleep(2)
        mqtt.unsubscribe(topic)

        return jsonify({
            'message': 'No Error'

```

Publish lệnh kèm topic đã được định nghĩa ở phần 2.3 tới broker

- Nếu lệnh hoặc topic không hợp lệ sẽ Response: 500
- Nếu hợp lệ sẽ Response 200

Dữ liệu sẽ được trả về

```

    }), 200
except Exception as e:
    return jsonify(f"Error querying devices: {e}"), 500

```

qua Websocket

2.7 Điều khiển device

```

@app.route('/api/v1/device/publish_cmd', methods=['POST'])
def publish_cmd():
    try:
        data = request.get_json()
        print(data)
        topic = topics_publish[data['topic']]
        cmd = commands[data['cmd']]
        mqtt.publish(topic, cmd)
        time.sleep(3)
        mqtt.unsubscribe(topic)

        return jsonify({
            'message': 'No Error'
        }), 200
    except Exception as e:
        # return {'error': e}, 500
        return {'error': 'invalid command'}, 500

```

Publish lệnh kèm topic đã được định nghĩa ở phần 2.3 tới broker

- Nếu lệnh hoặc topic không hợp lệ sẽ Response: 500
- Nếu hợp lệ sẽ Response 200

Dữ liệu sẽ được trả về qua Websocket

2.8 Lấy lịch sử điều khiển device

```

@app.route('/api/v1/device/logs', methods=['POST'])
def device_logs():
    try:
        data = request.get_json()
        page = data.get('page', 1)
        per_page = data.get('per_page', 10)
        latest = data.get('latest', True)
        timestamp = data.get('timestamp', None)
        fromDay = data.get('fromDay', False)
        toDay = data.get('toDay', False)

        query = DeviceHistory.query

        # Sắp xếp và phân trang kết quả
        if latest:
            query = query.order_by(DeviceHistory.timestamp.desc())
        else:
            query = query.order_by(DeviceHistory.timestamp.asc())

        if timestamp is not None:
            query = query.filter(DeviceHistory.timestamp.like(f"{timestamp}%"))

        data_paginated = query.paginate(page=page, per_page=per_page,
error_out=False)

        res = [data.to_dict() for data in data_paginated]
        return jsonify({
            'page': data_paginated.page,
            'per_page': data_paginated.per_page,
            'total': data_paginated.total,

```

```

        'pages': data_paginated.pages,
        'data': res
    }), 200
except Exception as e:
    print(str(e))
    return jsonify({'error': str(e)}), 500

```

2.9 Lấy lịch sử ghi lại data sensor

```

@app.route('/api/v1/sensors/streaming/logs', methods=['POST'])
def data_stream_logs():
    try:
        data = request.get_json()
        page = data.get('page', 1)
        per_page = data.get('per_page', 10)
        latest = data.get('latest', True)

        temp = data.get('temp', None)
        humidity = data.get('humidity', None)
        light = data.get('light', None)
        timestamp = data.get('timestamp', None)

        fromDay = data.get('fromDay', False)
        toDay = data.get('toDay', False)

        query = DataRealTime.query

        # Sắp xếp và phân trang kết quả
        if latest:
            query = query.order_by(DataRealTime.timestamp.desc())
        else:
            query = query.order_by(DataRealTime.timestamp.asc())

        if temp is not None:
            query = query.filter(DataRealTime.temp.like(f"{temp}%"))
        if humidity is not None:
            query = query.filter(DataRealTime.humidity.like(f"{humidity}%"))
        if light is not None:
            query = query.filter(DataRealTime.light.like(f"{light}%"))
        if timestamp is not None:
            query = query.filter(DataRealTime.timestamp.like(f"{timestamp}%"))

        data_paginated = query.paginate(page=page, per_page=per_page,
error_out=False)

        res = [data.to_dict() for data in data_paginated]
        return jsonify({
            'page': data_paginated.page,
            'per_page': data_paginated.per_page,
            'total': data_paginated.total,
            'pages': data_paginated.pages,
            'data': res
        }), 200
    except Exception as e:
        return jsonify({'error': str(e)}), 500

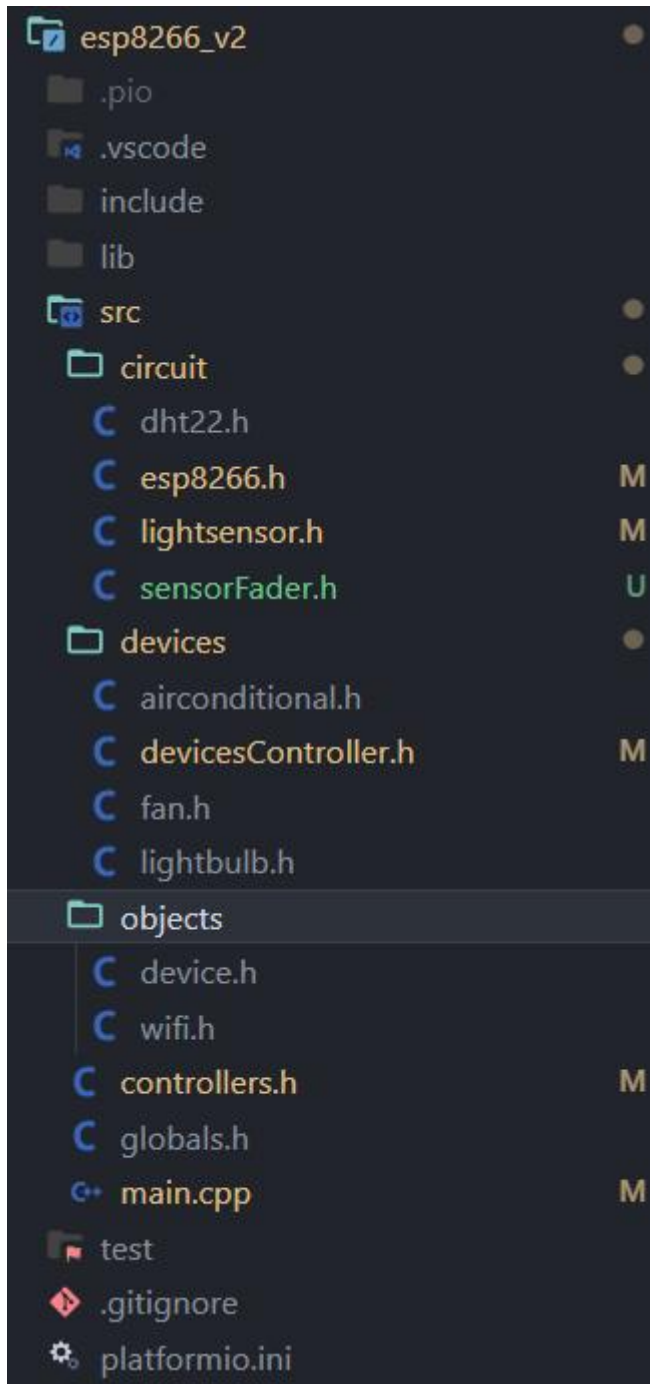
```

2.10 Khởi chạy server

```
''' -----SERVER LAUNCH-----  
'''  
  
if __name__ == '__main__':  
    # app.run(port=PORT, debug=True)  
  
    with app.app_context():  
        db.create_all()  
        countWarning = WarningSensorCount.query.count()  
        if countWarning == 0:  
            try:  
                for sensor in sensors.values(): # Sử dụng .values() để lặp qua  
giá trị của từ điển  
                    new_warning_count =  
WarningSensorCount(sensor_name=sensor['name'], count=0)  
                    db.session.add(new_warning_count)  
                    # Commit tất cả các thay đổi một lần sau vòng lặp để tăng hiệu  
suất  
                    db.session.commit()  
                    print("WarningSensorCount created successfully")  
            except Exception as e:  
                print("Error creating WarningSensorCount: ", e)  
  
    socketio.run(app, host='0.0.0.0', port=5000, use_reloader=True, debug=True)
```

3. Arduino

3.1 Cấu trúc thư mục



3.2 Khai báo thư mục và các biến toàn cục

```
// globals.h
#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <PubSubClient.h>

// -----DEFINE Data Global-----
//
extern char receivedTopic_global[100];
extern char receivedPayload_global[256];
```

```
// -----DEFINE COMMAND-----
// -----
extern char cmd[2][5];
extern char topic_subscribes[10][256];
extern char topic_publish[10][256];
```

3.3 Định nghĩa đối tượng ESP8266

```
#include "../globals.h"
#include "../objects/wifi.h"

// Định nghĩa các biến toàn cục đã khai báo trong globals.h
char receivedTopic_global[100];
char receivedPayload_global[256];
char cmd[2][5];
char topic_subscribes[10][256];
char topic_publish[10][256];
class Esp8266
{
private:
    // Thông tin MQTT Broker
    // Địa chỉ IP của MQTT broker
    const char *mqtt_server = "192.168.1.16";
    const int mqtt_port = 1884;
    // Tài khoản MQTT
    const char *mqtt_user = "ngn";
    const char *mqtt_pass = "ngn";
    Wifi wifi = Wifi();
    WiFiClient wifiClient;
public:
    PubSubClient psClient;
    // Khởi tạo PubSubClient với WiFiClient bằng initializer list
    Esp8266() : psClient(wifiClient)
    {
        // Gán giá trị cho các biến toàn cục đã được định nghĩa bằng 'extern'
        strcpy(cmd[0], "ON");
        strcpy(cmd[1], "OFF");
        strcpy(topic_subscribes[0], "fan/pub");
        strcpy(topic_subscribes[1], "airConditioner/pub");
        strcpy(topic_subscribes[2], "lightBulb/pub");
        strcpy(topic_subscribes[3], "allDevice/pub");
        strcpy(topic_subscribes[4], "deviceStatus/pub");
        strcpy(topic_publish[0], "fan/sub");
        strcpy(topic_publish[1], "airConditioner/sub");
        strcpy(topic_publish[2], "lightBulb/sub");
        strcpy(topic_publish[3], "allDevice/sub");
        strcpy(topic_publish[4], "deviceStatus/sub");
    }
    void connect_wifi()
    {
        wifi.connect_wifi();
    }
    // Hàm callback khi có thông điệp mới từ một topic MQTT
    void callback(char *topic, byte *payload, unsigned int length)
    {
        // Lưu trữ topic vào biến global
        strncpy(receivedTopic_global, topic, sizeof(receivedTopic_global) - 1);
        receivedTopic_global[sizeof(receivedTopic_global) - 1] = '\0';
        Serial.print("-----");
        Serial.print("Callback receive a topic: ");
        Serial.println(topic);
    }
};
```

```

Serial.print("Message: ");
for (unsigned int i = 0; i < length; i++)
{
    Serial.print((char)payload[i]);
    // Lưu trữ message vào global
    receivedPayload_global[i] = (char)payload[i];
}
receivedPayload_global[length] = '\0';
Serial.println();
Serial.println("-----");
}
void reconnect()
{
    // Loop until we're reconnected
    while (!psClient.connected())
    {
        Serial.print("Attempting MQTT connection...");
        // Tạo một clientId ngẫu nhiên
        String clientId = "ESP8266Client-";
        clientId += String(random(0xffff), HEX);
        // Thử kết nối
        if (psClient.connect(clientId.c_str(), mqtt_user, mqtt_pass))
        {
            Serial.println("connected");
            // Khi kết nối thành công, gửi một thông báo
            psClient.publish("test_topic", "hello world");
            // Và đăng ký lại để nhận thông báo từ topic
            psClient.subscribe("test_topic");
        }
        else
        {
            Serial.print("failed, rc=");
            Serial.print(psClient.state());
            Serial.println(" try again in 5 seconds");
            // Hiện thị chi tiết lỗi
            switch (psClient.state())
            {
                case -4:
                    Serial.println("Connection timeout");
                    break;
                case -3:
                    Serial.println("Connection lost");
                    break;
                case -2:
                    Serial.println("Connect failed");
                    break;
                case -1:
                    Serial.println("Disconnected");
                    break;
                case 1:
                    Serial.println("Bad protocol version");
                    break;
                case 2:
                    Serial.println("Bad client ID");
                    break;
                case 3:
                    Serial.println("Unavailable");
                    break;
                case 4:
                    Serial.println("Bad credentials");
                    break;
            }
        }
    }
}

```

```

        case 5:
            Serial.println("Unauthorized");
            break;
        default:
            Serial.println("Unknown error");
    }
    // Đợi 5 giây trước khi thử lại
    delay(5000);
}
}
}

void publish(String topic, String message)
{
    // Publish một thông điệp đến topic
    Serial.print("Publishing topic: " + topic + " -> data: ");
    Serial.println(message);
    psClient.publish(topic.c_str(), message.c_str());
}

void set_connection_broker()
{
    // Kết nối đến MQTT broker
    psClient.setServer(mqtt_server, mqtt_port);
    psClient.setCallback(
        [this](char *topic, byte *payload, unsigned int length)
        {
            this->callback(topic, payload, length);
        });
    /*
     * Thay vì gọi hàm ngay lập tức, cần truyền một con trỏ hàm (function pointer) để khi có tin
     * nhắn đến,
     * hàm callback sẽ được gọi.
     */
}
};

```

3.4 Định nghĩa cảm biến DHT22

```

#include <DHT.h>

#define DHTPIN 14
#define DHTTYPE DHT22 // Chọn loại cảm biến DHT22
DHT dht22(DHTPIN, DHTTYPE);

```

3.5 Định nghĩa cảm biến ánh sáng

```

#include "globals.h"

class LightSensor
{
private:
    /*
     * Chân GPIO để kết nối với cảm biến ánh sáng
     */
    uint8_t dataPin;
    /*
     * Giá trị đọc từ cảm biến
     */
    int lightValue;
public:
    /*
     * Constructor để khởi tạo cảm biến với chân GPIO
     */
}

```

```

    */
    LightSensor(uint8_t pin)
    {
        dataPin = pin;
        lightValue = 0;
    }
    /*
    * Cấu hình chân digital đầu vào cho cảm biến
    */
    void setPinMode()
    {
        pinMode(dataPin, INPUT);
    }
    /*
    * Đọc giá trị digital từ cảm biến ánh sáng
    */
    float readLight()
    {
        lightValue = analogRead(dataPin);
        return lightValue;
    }
    /*
    * In ra giá trị của cảm biến ánh sáng
    */
    void printLightValue()
    {
        Serial.print("Light sensor value: ");
        Serial.println(lightValue);
    }
};

```

3.6 Định nghĩa đối tượng Device

```

#include "globals.h"

#pragma once
class Device
{
protected:
    uint8_t PinMode;
public:
    boolean deviceIsActive = false;
    void turnOn()
    {
        digitalWrite(PinMode, HIGH);
        deviceIsActive = true;
    }
    void turnOff()
    {
        digitalWrite(PinMode, LOW);
        deviceIsActive = false;
    }
    void isAlreadyOn()
    {
    }
    void isAlreadyOff()
    {
    }
    boolean isActive()
    {
        return deviceIsActive;
    }
};

```

```

    }
};
// #endif // Kết thúc include guard DEVICE_H

```

3.7 Định nghĩa đối tượng Wifi

```

#include "globals.h"

class Wifi
{
private:
    // Thông tin kết nối Wi-Fi
    const char *ssid = "0x6e676e"; // Thay bằng tên mạng Wi-Fi của bạn
    const char *password = "ngn@0x6e676e"; // Thay bằng mật khẩu Wi-Fi của bạn
public:
    void connect_wifi()
    {
        // Kết nối tới Wi-Fi
        Serial.println();
        Serial.print("Connecting to ");
        Serial.println(ssid);
        WiFi.begin(ssid, password);
        while (WiFi.status() != WL_CONNECTED)
        {
            delay(500);
            Serial.print(".");
        }
        Serial.println("");
        Serial.println("WiFi connected");
        Serial.println("IP address: ");
        Serial.println(WiFi.localIP());
    }
};

```

3.8 Định nghĩa chân đèn D1 tương trưng cho thiết bị Quạt

```

#include "../objects/device.h"

class Fan : public Device
{
public:
    Fan()
    {
        PinMode = D1;
    }
};

```

3.9 Định nghĩa chân đèn D2 tương trưng cho thiết bị điều hòa

```

#include "../objects/device.h"

class AirConditional : public Device
{
public:
    AirConditional()
    {
        PinMode = D2;
    }
};

```

3.10 Định nghĩa chân đèn D3 tương trưng cho thiết bị lightbulb

```

#include "../objects/device.h"

```

```

class LightBulb : public Device
{
public:
    LightBulb()
    {
        PinMode = D3;
    }
};

```

3.11 Đối tượng điều khiển thiết bị

```

#include "../globals.h"
#include "fan.h"
#include "lightbulb.h"
#include "airconditional.h"

struct ResultProcessTopic
{
    String topic;
    int isActive;
};

String prepare_topic_to_publish(String topicReceived);
class DevicesController
{
private:
    Fan fan = Fan();
    LightBulb lightBulb = LightBulb();
    AirConditional airConditional = AirConditional();
    boolean allDeviceIsActive = false;
    /*
Hàm xử lý bật tắt 1 device
*/
    unsigned int device_process(Device *device)
    {
        unsigned int isActive = -1;
        // nếu device đang tắt và subscribe cmd là ON thì cho bật device
        if (device->deviceIsActive == false && strcmp(receivedPayload_global, cmd[0]) == 0)
        {
            isActive = 1;
            device->turnOn();
        }
        // nếu device đang bật và subscribe cmd là OFF thì cho tắt device
        else if (device->deviceIsActive == true && strcmp(receivedPayload_global, cmd[1]) == 0)
        {
            isActive = 0;
            device->turnOff();
        }
        /*
        *
        */
        if (this->allDeviceIsActive == true)
            this->allDeviceIsActive = false;
    }
    // Nếu device đang tắt và subscribe cmd là OFF thì thông báo lại
    else if (device->deviceIsActive == false && strcmp(receivedPayload_global, cmd[1]) == 0)
    {
        // không cho tương tác với device
        isActive = 2;
        device->isAlreadyOff();
    }
    // nếu device đang bật và subscribe cmd là ON thì thông báo lại
    else if (device->deviceIsActive == true && strcmp(receivedPayload_global, cmd[0]) == 0)
    {

```

```

        // không cho tương tác với device
        isActive = 3;
        device->isAlreadyOn();
    }
    return isActive;
}
unsigned int all_devices_process()
{
    unsigned int isActive = -1;
    allDeviceIsActive = this->fan.isActive() && this->airConditional.isActive() && this->lightBulb.isActive()
        ? true
        : false;
    // nếu tắt cả device đang tắt và subscribe cmd là ON thì cho bật device
    if (allDeviceIsActive == false && strcmp(receivedPayload_global, cmd[0]) == 0)
    {
        allDeviceIsActive = true;
        isActive = 1;
        this->fan.turnOn();
        this->lightBulb.turnOn();
        this->airConditional.turnOn();
    }
    // nếu tắt device đang bật và subscribe cmd là OFF thì cho tắt device
    else if (allDeviceIsActive == true && strcmp(receivedPayload_global, cmd[1]) == 0)
    {
        isActive = 0;
        allDeviceIsActive = false;
        this->fan.turnOff();
        this->lightBulb.turnOff();
        this->airConditional.turnOff();
    }
    // Nếu device đang tắt và subscribe cmd là OFF thì thông báo lại
    else if (allDeviceIsActive == false && strcmp(receivedPayload_global, cmd[1]) == 0)
    {
        // không cho tương tác với device
        isActive = 2;
    }
    // nếu device đang bật và subscribe cmd là ON thì thông báo lại
    else if (allDeviceIsActive == true && strcmp(receivedPayload_global, cmd[0]) == 0)
    {
        // không cho tương tác với device
        isActive = 3;
    }
    if (fan.deviceIsActive == true && airConditional.deviceIsActive == true && lightBulb.deviceIsActive == true)
    {
        allDeviceIsActive = true;
    }
    else if (fan.deviceIsActive == false && airConditional.deviceIsActive == false && lightBulb.deviceIsActive == false)
    {
        allDeviceIsActive = false;
    }
    return isActive;
}
unsigned int getDeviceStatus(String &status)
{
    unsigned int isActive = 4;
    status = "{" +
        String("\"fan\": ") + (this->fan.deviceIsActive ? "true" : "false") + "," +
        String("\"airConditional\": ") + (this->airConditional.deviceIsActive ? "true" :
"false") + "," +

```

```

        String("\lightBulb\": ") + (this->lightBulb.deviceIsActive ? "true" : "false") + "," +
        String("\allDevice\": ") + (this->allDeviceIsActive ? "true" : "false") +
        "}";
    return isActive;
}
public:
void process_topic(PubSubClient &psClient)
{
    ResultProcessTopic result;
    /*
     * Kết quả xử lý Topic được lưu vào status
     */
    String status = "";
    /*
     * Define: isActive
     * 0
     * 1
     */
    unsigned int isActive = -2;
    /*
     * Nếu không có Topic nào gửi đến thì không xử lý
     */
    if (strlen(receivedTopic_global) == 0)
    {
        result.topic = "NoTopicReceived";
        result.isActive = -2;
        return;
    }
    Serial.print("Topic nhận được: ");
    Serial.println(receivedTopic_global);
    Serial.print("Payload nhận được: ");
    Serial.println(receivedPayload_global);
    /*
     * Router Xử Lý Subscribe Topic
     */
    /*
     * Nếu topic này điều khiển cả 3 thiết bị
     */
    if (strcmp(receivedTopic_global, topic_subscribes[3]) == 0)
    {
        isActive = this->all_devices_process();
    }
    /*
     * Nếu topic này là fan
     */
    else if (strcmp(receivedTopic_global, topic_subscribes[0]) == 0)
    {
        isActive = this->device_process(&fan);
    }
    /*
     * Nếu topic này là điều hòa
     */
    else if (strcmp(receivedTopic_global, topic_subscribes[1]) == 0)
    {
        isActive = this->device_process(&airConditional);
    }
    /*
     * Nếu topic này là đèn
     */
    else if (strcmp(receivedTopic_global, topic_subscribes[2]) == 0)
    {
        isActive = this->device_process(&lightBulb);
    }
}

```

```

}
/*
 * Nếu topic này là lấy trạng thái ban đầu của device
 */
else if (strcmp(receivedTopic_global, topic_subscribes[4]) == 0)
{
    isActive = this->getDeviceStatus(status);
}
/*
 * Trả kết quả xử lý topic
 */
result.topic = receivedTopic_global;
result.isActive = isActive;
/*
 * chuyển topic được publish đến thành dạng subscribe của server, sau đó publish nó đi.
 * Ví dụ: fan/pub -> fan/sub.
 */
String topic_to_publish = prepare_topic_to_publish(result.topic);
/*
 * Xử lý Publish Topic
 */
/*
 * Nếu có topic hợp lệ gửi đến và topic này đã được định nghĩa đúng format : fan/pub
 */
if (topic_to_publish != "Invalid delimiter")
{
    /*
     * Bật thành công
     */
    if (result.isActive == 1)
    {
        status = "TurnOn Successfully";
        Serial.println("TurnOn successfully");
        delay(100);
    }
    /*
     * Tắt thành công
     */
    else if (result.isActive == 0)
    {
        status = "TurnOff Successfully";
        Serial.println("TurnOff successfully");
        delay(100);
    }
    /*
     * nếu đang tắt mà Publish đến lệnh tắt
     */
    else if (result.isActive == 2)
    {
        status = "is already off!";
        Serial.println(status);
        delay(100);
    }
    /*
     * nếu đang bật mà Publish đến lệnh bật
     */
    else if (result.isActive == 3)
    {
        status = "is already on!";
        Serial.println(status);
        delay(100);
    }
}

```

```

        /*
        * Trạng thái trả về của get status device
        */
        else if (result.isActive == 4)
        {
        }
        /*
        * Nếu lệnh được Publish đến bị sai logic nên không được thực hiện
        */
        else if (result.isActive == -1)
        {
            status = "Wrong Command";
            Serial.println("Failed to active message");
            delay(100);
        }
    }
    /*
    * Tạo Chuỗi JSON để gửi đi
    */
    String message = "{\"topic\":\"" + topic_to_publish + "\",\" " +
                    "\"cmd\":\"" + receivedPayload_global + "\",\" " +
                    "\"status\":\"" + status + "\"}";
    Serial.println("Tạo Chuỗi JSON để gửi đi");
    Serial.println(message);
    /*
    * Xoá topic và payload sau khi hoàn thành
    */
    strncpy(receivedTopic_global, "", sizeof(receivedTopic_global) - 1);
    receivedTopic_global[0] = '\0';
    strncpy(receivedPayload_global, "", sizeof(receivedPayload_global) - 1);
    receivedPayload_global[0] = '\0';
    psClient.publish(topic_to_publish.c_str(), message.c_str());
}
};
// ----- String prepare_topic_to_publish(String topicReceived) -----
// ----- //
String prepare_topic_to_publish(String topicReceived)
{
    String delimiter = "/"; // Ký tự phân tách
    int delimiterIndex = topicReceived.indexOf(delimiter);
    if (delimiterIndex != -1)
    {
        /*
        * Trả về phần chuỗi trước ký tự phân tách
        */
        String topic_name = topicReceived.substring(0, delimiterIndex);
        return topic_name + "/sub";
    }
    else
    {
        /*
        * Nếu không có ký tự phân tách, trả về invalid
        */
        return "Invalid delimiter";
    }
}
// -----END-----
// ----- //

```

3.12 Đối tượng điều khiển luồng hệ thống

```
#include "globals.h"
#include "circuit/esp8266.h"
#include "circuit/dht22.h"
#include "circuit/lightsensor.h"
#include "devices/devicesController.h"
#include "circuit/sensorFader.h"

// -----DEFINE DATA TYPE-----
// -----
struct DataStreaming
{
    String topic;
    float temp;
    float humidity;
    int light;
    int dust;
    int rain;
    int windSpeed;
    String time;
    String dump;
};

// -----DEFINE COMMAND Controller-----
// -----
class Controller
{
private:
    DataStreaming dataStreaming;
    /*
     * Khởi tạo cảm biến ánh sáng nối với chân GPIO12 (D6)
     */
    LightSensor lightSensor = LightSensor(A0);
    SensorFader sensorFader = SensorFader();
public:
    Esp8266 esp;
    DevicesController devicesController;
    /*
     * Khởi tạo lớp Controller và khởi tạo đối tượng ESP
     */
    Controller() : esp(), devicesController()
    {
        // connect wifi
        esp.connect_wifi();
        // connect broker MQTT
        esp.set_connection_broker();
        // Khởi động cảm biến DHT22
        dht22.begin();
    }
    void set_init_state_pin_mode()
    {
        /*
         * Đặt chế độ cho các chân D1, D2, D3, D5, D6 làm OUTPUT để điều khiển đèn LED
         */
        lightSensor.setPinMode();
        pinMode(D1, OUTPUT);
        pinMode(D2, OUTPUT);
        pinMode(D3, OUTPUT);
        digitalWrite(D1, LOW); // Đèn ban đầu tắt
        digitalWrite(D2, LOW); // Đèn ban đầu tắt
        digitalWrite(D3, LOW); // Đèn ban đầu tắt
    }
}
```

```

// ----- Start Streaming Func -----
//
void Streaming()
{
    DataStreaming streaming;
    streaming.topic = "streaming/all";
    /*
     * Đọc giá trị nhiệt độ và độ ẩm
     */
    float humidity = dht22.readHumidity();
    float temperature = dht22.readTemperature();
    float light = lightSensor.readLight();
    Serial.print("Light sensor value: ");
    Serial.println(light);
    // Kiểm tra xem có đọc được giá trị hợp lệ không
    if (isnan(humidity) || isnan(temperature) || isnan(light))
    {
        Serial.println("Không đọc được dữ liệu từ cảm biến DHT22!");
        return;
    }
    streaming.temp = temperature;
    streaming.humidity = humidity;
    streaming.light = 1024 - light;
    streaming.dust = sensorFader.readDust();
    streaming.rain = sensorFader.readRain();
    streaming.windSpeed = sensorFader.readWindSpeed();
    streaming.dump = "{\"temp\": " + String(streaming.temp) + "," +
        "\"humidity\": " + String(streaming.humidity) + "," +
        "\"light\": " + String(streaming.light) + "," +
        "\"dust\": " + String(streaming.dust) + "," +
        "\"rain\": " + String(streaming.rain) + "," +
        "\"windSpeed\": " + String(streaming.windSpeed) + "}";
    esp.publish(streaming.topic, streaming.dump);
}
// -----END Streaming Func-----
//
};

```

3.13 Hàm main của chương trình

```

#include "globals.h"
#include "controllers.h"

unsigned long previousMillis = 0;
const long interval = 2000; // 2000 giây
// Khai báo con trỏ controller
Controller *controller;
void setup()
{
    Serial.begin(115200);
    Serial.println("Setup started");
    delay(1000);
    /*
     * Tạo đối tượng controller
     * Kết nối wifi
     * set connect tới broker
     * khởi động dht22
     */
    controller = new Controller();
}

```

```

/*
 * Đặt chế độ cho các chân D1, D2, D3 làm OUTPUT để điều khiển đèn LED
 */
controller->set_init_state_pin_mode();
}
void loop()
{
  if (!controller->esp.psClient.connected())
  {
    controller->esp.reconnect();
    controller->esp.psClient.subscribe("fan/pub");
    controller->esp.psClient.subscribe("airConditioner/pub");
    controller->esp.psClient.subscribe("lightBulb/pub");
    controller->esp.psClient.subscribe("allDevice/pub");
    controller->esp.psClient.subscribe("deviceStatus/pub");
  }
  /*
   * Gọi psClient.loop() để xử lý gói tin MQTT
   */
  controller->esp.psClient.loop();
  /*
   * Xử lý dữ liệu và nhận dữ liệu từ các topic subscribed
   */
  controller->devicesController.process_topic(controller->esp.psClient);
  /*
   * Xử lý dữ liệu và publish định kỳ
   */
  unsigned long currentMillis = millis();
  if (currentMillis - previousMillis >= interval)
  {
    previousMillis = currentMillis;
    controller->Streaming();
  }
}
}

```

4. ReactJs

4.1 Websocket

```

import React, { useState, useEffect } from 'react'
import { socket } from '../../socket'
import { useDispatch } from 'react-redux'
import { addDataStream } from '../../redux/Streaming'
import { setResponseCommand } from '../../redux/ResponseCommand'
import { setDeviceStatus } from '../../redux/DeviceStatus'
import { setStateWarningCount } from '../../redux/WaringCountRedux'

export default function WebSocketComponent() {
  const [isConnected, setIsConnected] = useState(socket.connected)
  const [fooEvents, setFooEvents] = useState([])
  const dispatch = useDispatch()
  useEffect(() => {
    function onConnect() {
      console.log('Connected to the server')
      setIsConnected(true)
      const message = 'hello server!'
      console.log('client send: ', message)
      socket.emit('message', message)
    }
    function onDisconnect() {
      setIsConnected(false)
    }
  })
}

```

```

    }
    function onResponse(value) {
        console.log(value)
    }
    function onWellComeEvent(value) {
        console.log('received welcome event:', value)
        setFooEvents(previous => [...previous, value])
    }
    function onStreamingAll(value) {
        // console.log('received streaming/all:', value)
        dispatch(addDataStream(value))
    }
    function onDevice(value) {
        dispatch(setResponseCommand(value))
    }
    function onDeviceStatus(value) {
        dispatch(setDeviceStatus(value))
    }
    function onWaringCount(value) {
        dispatch(setStateWarningCount(value))
    }
    socket.on('connect', onConnect)
    socket.on('response', onResponse)
    socket.on('streaming/all', onStreamingAll)
    socket.on('device', onDevice)
    socket.on('deviceStatus', onDeviceStatus)
    socket.on('waring_count', onWaringCount)
    return () => {
        socket.off('connect', onConnect)
        socket.off('disconnect', onDisconnect)
        socket.off('welcome', onWellComeEvent)
        socket.off('mqtt_message', onStreamingAll)
        // socket.disconnect()
    }
}, [dispatch])
return (
    <></>
)
}

```

4.2 Lấy trạng thái ban đầu của device

```

useEffect(() => {
    axios.get(config.backend.baseUrl + '/device/status')
    axios.get(config.backend.baseUrl + '/device/status')
}, []);

```

4.3 Lấy lịch sử điều khiển thiết bị và filter dữ liệu

```

useEffect(() => {
    const fetchData = () => {
        axios.post(config.backend.baseUrl + '/device/logs', {
            "page": currentPage,
            "per_page": perPage,
            "latest": isLatest,
            "fromDay": false,
            "toDay": false,
            "timestamp": filterTimestamp,
        })
    }
    .then(response => {

```

```

        setData(response.data || { data: [] }) // Nếu không có data, trả về mảng rỗng
        console.log(response.data)
    })
    .catch(error => {
        console.error(error)
    })
    .finally(() => {
        setIsLoading(false)
    })
}
fetchData()
}, [currentPage, perPage, isLatest, fromDay, toDay, filterTimestamp])

```

4.4 Lấy lịch sử dữ liệu sensor và filter dữ liệu

```

useEffect(() => {
    const fetchData = () => {
        axios.post(config.backend.baseUrl + '/sensors/streaming/logs', {
            "page": currentPage,
            "per_page": perPage,
            "latest": isLatest,
            // "fromDay": fromDay,
            // "toDay": toDay,
            "fromDay": false,
            "toDay": false,
            "temp": filterTemp,
            "humidity": filterHumidity,
            "light": filterLight,
            "timestamp": filterTimestamp,
        })
        .then(response => {
            setData(response.data);
            console.log(response.data);
        })
        .catch(error => {
            console.error(error);
        })
        .finally(() => {
            setIsLoading(false);
        });
    };

    fetchData(); // Gọi hàm fetchData ngay khi component mount hoặc khi dependencies thay đổi
    if (isRealTime) {
        const interval = setInterval(fetchData, 2000); // Lặp lại sau mỗi 2 giây
        return () => clearInterval(interval);
    }
}, [currentPage, perPage, isRealTime, isLatest, fromDay, toDay, filterTemp, filterHumidity, filterLight, filterTimestamp]);

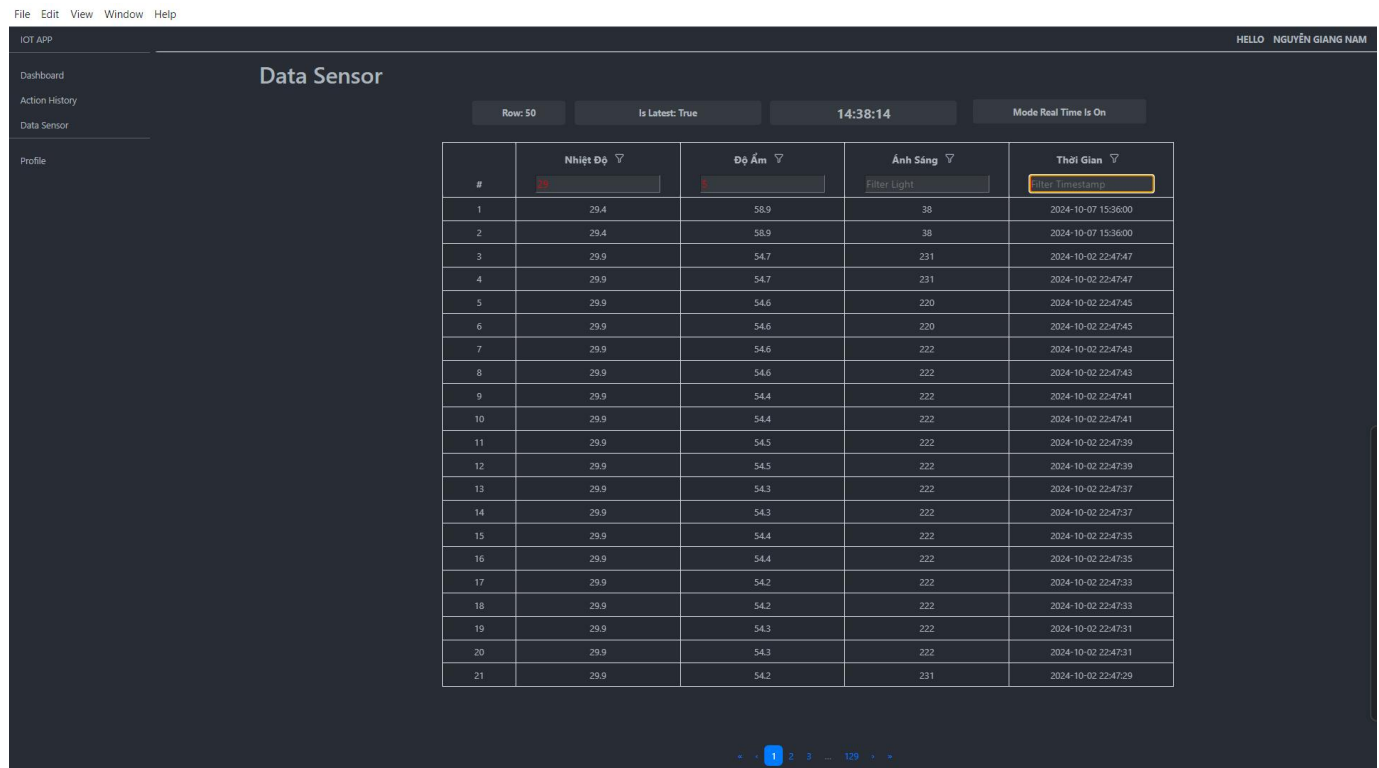
```

VI. Kết quả

1. Hiển thị dữ liệu DashBoard

- Dữ liệu Sensor qua Websocket được hiển thị lên đồ thị
- Bảng điều khiển bật/tắt các thiết bị
- Bảng ghi lại lịch sử cảnh báo sensor vượt ngưỡng

- Chức năng kết hợp tìm kiếm: nhiệt độ, độ ẩm, ánh sáng, thời gian



4. Trang Profile

